

Calcul d'adresses Amorçage

Sommaire

Allocation des données

[Portée des liaisons](#)

[Calcul des adresses](#)

[Amorçage](#)

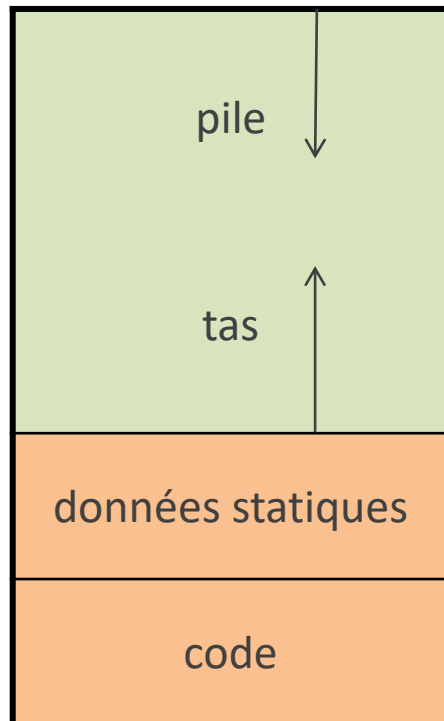
Organisation de la mémoire

adresses

hautes



basses



Allocation automatique

À l'entrée des blocs

Les adresses dans la pile sont des adresses relatives (*offsets*) par rapport à une certaine position dans la pile (base)

Allocation dynamique

Sur demande explicite dans le source

Allocation statique

Placement des variables statiques

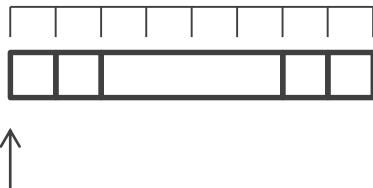
Nom	Type	Adresse relative
c1	char	0
c2	char	1
i	int	2
d1	char	6
d2	char	7

```
char c1,c2;
int i;
char d1,d2;
```

À la construction de la table des symboles
Attribuer les adresses relatives en tenant compte
des emplacements déjà occupés

Si les adresses relatives partent de 0 et sont
positives, la prochaine adresse relative
disponible est la taille occupée par les
variables allouées

On l'utilise pour allouer la mémoire



Base des adresses relatives

Placement des variables automatiques

Nom	Type	Adresse relative
c1	char	-1
c2	char	-2
i	int	-6
d1	char	-7
d2	char	-8

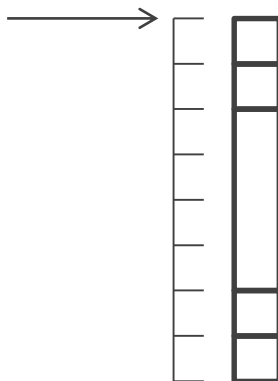
```
char c1,c2;
int i;
char d1,d2;
```

À la construction de la table des symboles
Attribuer les adresses relatives en tenant compte
des emplacements déjà occupés

Si les adresses relatives partent de 0 et sont
négatives, la dernière adresse relative
attribuée donne la taille occupée par les
variables allouées

On l'utilise pour allouer la mémoire

Base des
adresses
relatives



Alignement en mémoire

Avec gcc sous Linux

machine	type	long	long long	float	double	long double
32 bits	taille (octets)	4	8	4	8	12
	alignement	4	4	4	4	4
64 bits	taille	8	8	4	8	16
	alignement	8	8	4	8	16



Alignement : quand l'adresse d'une donnée doit être divisible par un certain entier

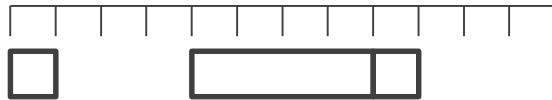
Sert à rendre plus efficaces les accès mémoire

Cela peut obliger le compilateur à laisser du remplissage (*padding*)

L'alignement dépend aussi du compilateur

Alignement en mémoire

```
char c;  
int count;  
char d;
```



S'il y a des contraintes d'alignement, on peut modifier la quantité de remplissage en jouant sur le placement des variables

On obtient le remplissage minimal en définissant les champs par ordre de tailles décroissantes

La lisibilité dépend aussi de l'ordre dans lequel on définit les champs

Sommaire

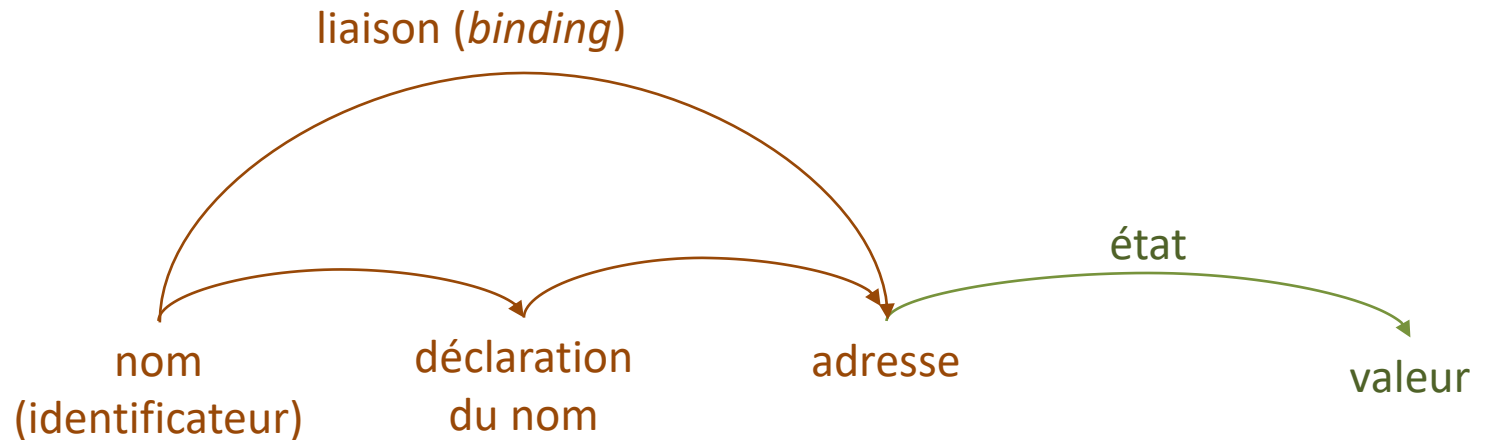
Allocation des données

Portée des liaisons

Calcul des adresses

Amorçage

Liaison d'un nom



Si l'environnement associe une occurrence du nom *id* à l'adresse *a*, on dit que *id* est lié à *a*

La valeur stockée à une adresse dépend de chaque exécution

Une adresse peut contenir plusieurs valeurs successivement

main()

```

{
  B0  int a = 0 ;
      int b = 0 ;
      {
        B1  int b = 1 ;
            {
              int a = 2 ;
              printf("%d %d", a, b) ;
            }
            {
              int b = 3 ;
              printf("%d %d", a, b) ;
            }
            printf("%d %d", a, b) ;
        }
      printf("%d %d", a, b) ;
}
  
```

Portée (scope) d'une liaison

Partie du code source où la liaison est active

Déclaration globale

Valable par défaut dans tout le source

Déclaration locale

Valable par défaut dans un bloc

Le nom déclaré est **local** au bloc

Un nom local à un bloc cache (*shadows*) un nom non local

main()

```
{
  B0  int a = 0 ;
      int b = 0 ;
      {
        B1  int b = 1 ;
          {
            B2  int a = 2 ;
                printf("%d %d", a, b) ;
            }
          {
            B3  int b = 3 ;
                printf("%d %d", a, b) ;
            }
          printf("%d %d", a, b) ;
        }
      printf("%d %d", a, b) ;
}
```

Structure de blocs

Bloc --> { Déclarations Instructions }

Les instructions peuvent contenir des blocs

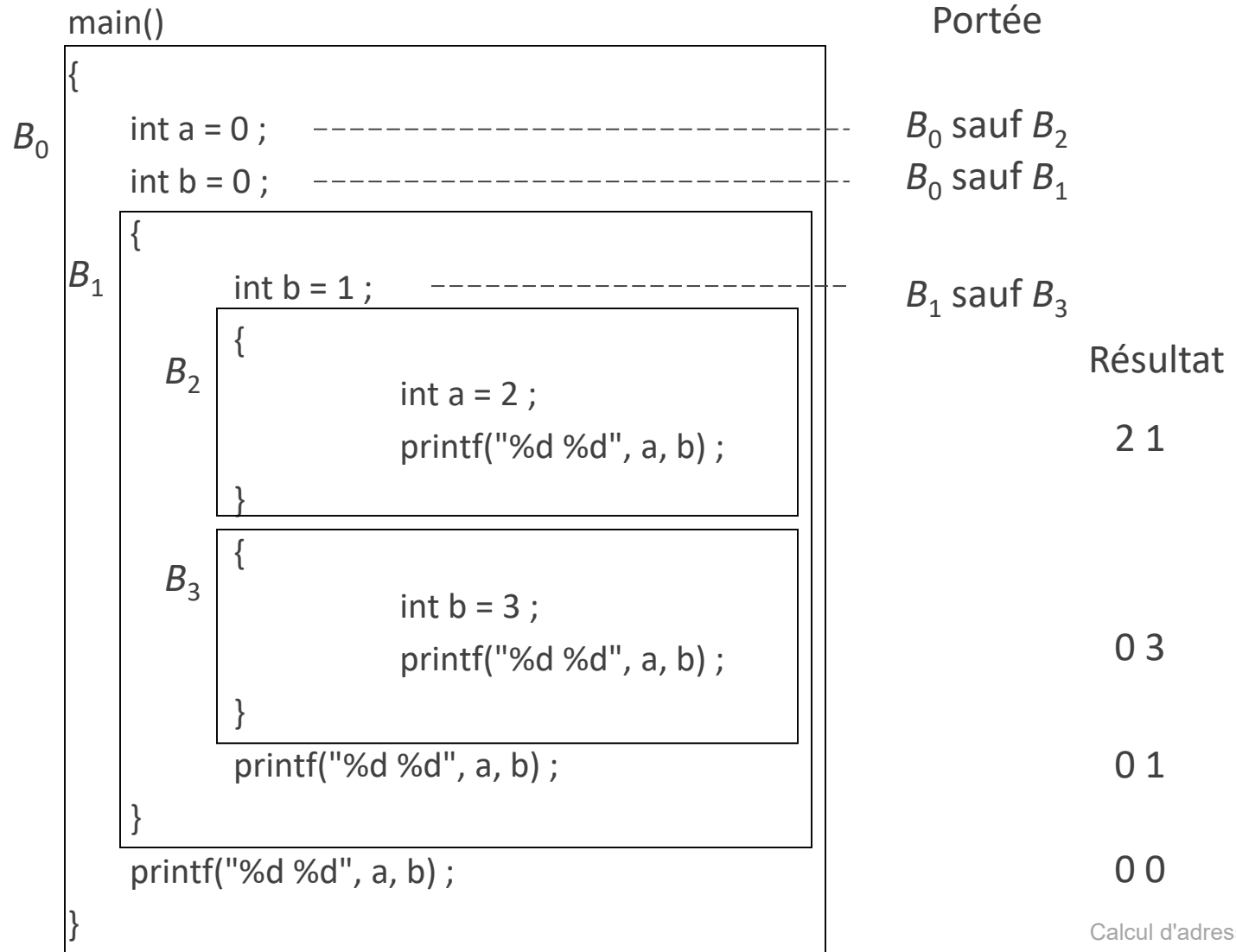
Une fonction peut contenir plusieurs blocs

Portée lexicale ou statique

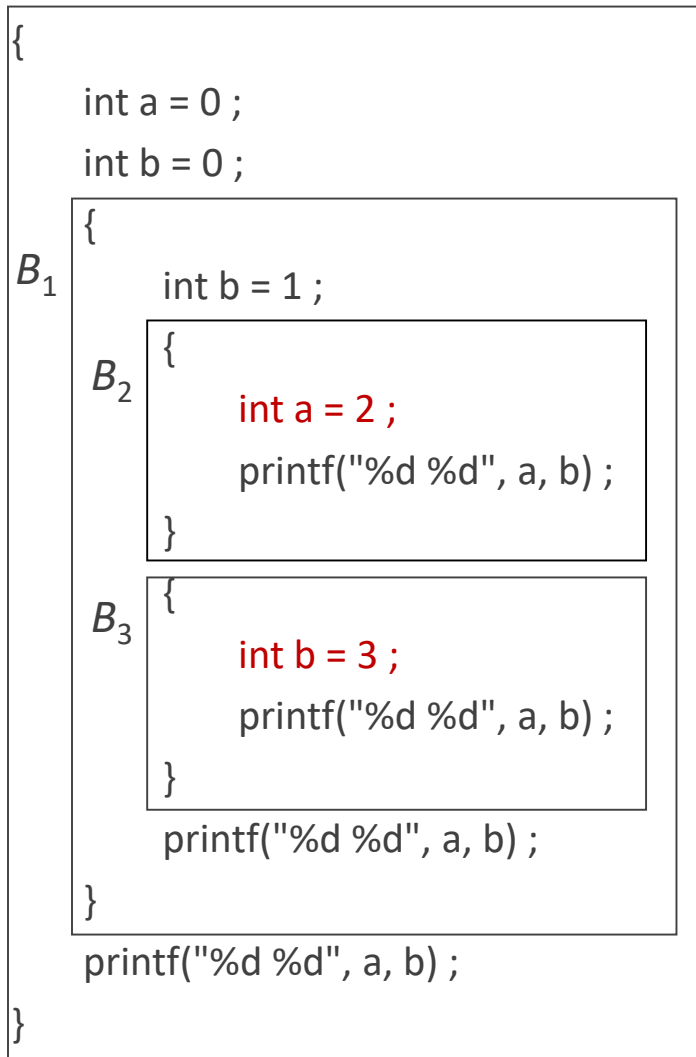
La portée d'une déclaration faite au début de B est incluse dans B

Si un nom x dans B n'est pas local à B , on cherche une déclaration de x au début d'un bloc B' qui contient B

S'il y en a plusieurs, on prend le plus petit



main()



Implémentation

Implémentation facile quand toutes les déclarations locales sont au début d'une fonction

Sinon : deux méthodes

Allocation par bloc

On traite chaque bloc comme une fonction sans paramètres ni valeur de retour

Allocation par fonction

On regroupe les variables déclarées dans toute la fonction

On peut lier à une même adresse deux variables dont les portées sont disjointes (a de B_2 et b de B_3)

Langages avec fonctions emboîtées

```
def pourcentages(a,b,c):
```

```
    def pc(x):
```

```
        B1 return (x*100.0)/(a+b+c)
```

```
    return pc(a),pc(b),pc(c)
```

Python, ADA, Perl, Lisp...

La fonction emboîtée est locale à la fonction dans laquelle elle est emboîtée

Par défaut, la portée de la fonction emboîtée est incluse dans le source de la fonction dans laquelle elle est emboîtée

Extension de la portée d'une liaison

Déclaration globale

Peut être étendue à d'autres modules par une
déclaration externe

Déclaration locale

Dans certains langages, peut être étendue à
d'autres blocs dynamiquement : portée
dynamique

Les règles de portée des variables dépendent du
langage source

Portée lexicale et portée dynamique

Portée lexicale ou statique

C, Java, Ada, Pascal

Portée déterminée par le code source du
programme

Ne change pas pendant l'exécution du programme

Portée dynamique

Perl, Lisp, APL, Snobol

Portée déterminée aussi par les activations en
cours

Change pendant l'exécution du programme


```
#!/usr/bin/perl
$r = 0.25;
sub show
{
    return $r;
}
sub small
{
    local $r = 0.125;
    return show();
}
print show()."\t".small()."\n";
print show()."\t".small()."\n";
```

Résultats

portée statique		portée dynamique	
0.25	0.25	0.25	0.125
0.25	0.25	0.25	0.125

Portée dynamique

Les liaisons des noms non locaux ne changent pas quand on appelle une fonction

Un nom non local dans la fonction appelée est lié au même emplacement que dans la fonction appelante

Perl, Lisp

Complicite le développement logiciel

Réalisation

On recherche l'emplacement des noms non locaux en suivant les liens de contrôle

Si le nom est local à une fonction définie dans un autre module, le calcul de l'adresse est fait à l'exécution

Noms non locaux

Différences entre langages

Nom non local : nom qui n'est pas local au bloc immédiatement englobant

Quels noms non locaux sont valables dans un bloc ?

En C :

- les noms locaux à un bloc englobant
- les noms globaux

En Ada : les noms valables dans la fonction englobante

En Lisp : les noms valables dans l'activation appelante

Allocation et portée

Allocation automatique

Les variables automatiques sont locales

Allocation statique

Les variables statiques ont généralement une déclaration globale

Exception : variables locales allouées statiquement

En C et C++ : déclaration locale statique

- Valeur est conservée au retour de la fonction
- Une même liaison pour plusieurs activations

En Java et Ada : pas de variables locales statiques, mais variables de classe en Java

Allocation et portée en C

Les noms sont de deux sortes :

- locaux à une fonction ou à un bloc
- globaux, déclarés hors des fonctions

Les noms globaux sont liés à des adresses
statiques

Les noms locaux sont liés à des adresses en pile,
accessibles à partir du pointeur de base...

sauf les noms locaux statiques

Sommaire

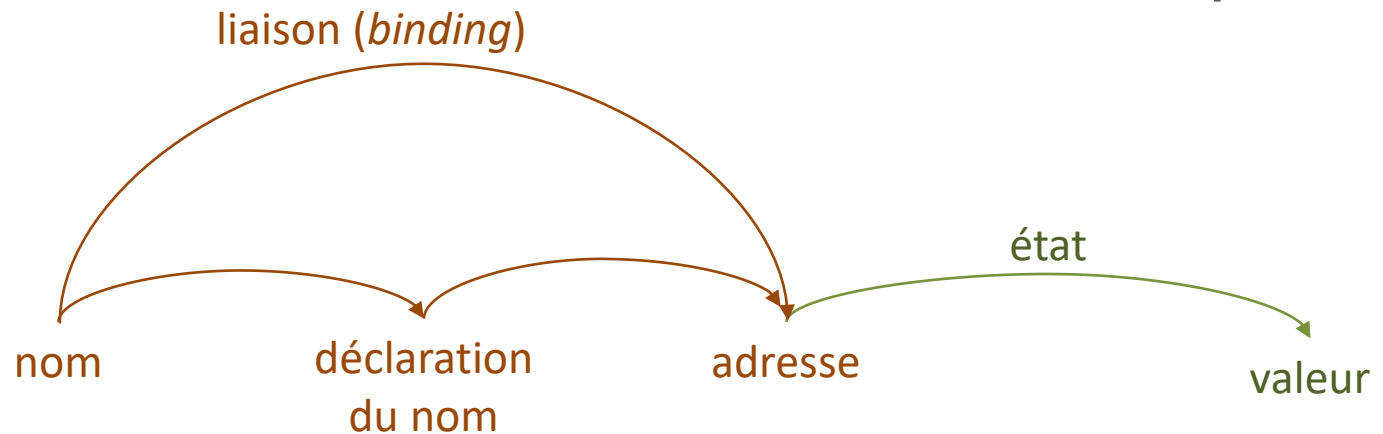
Allocation des données

Portée des liaisons

Calcul des adresses

Amorçage

Allocation statique



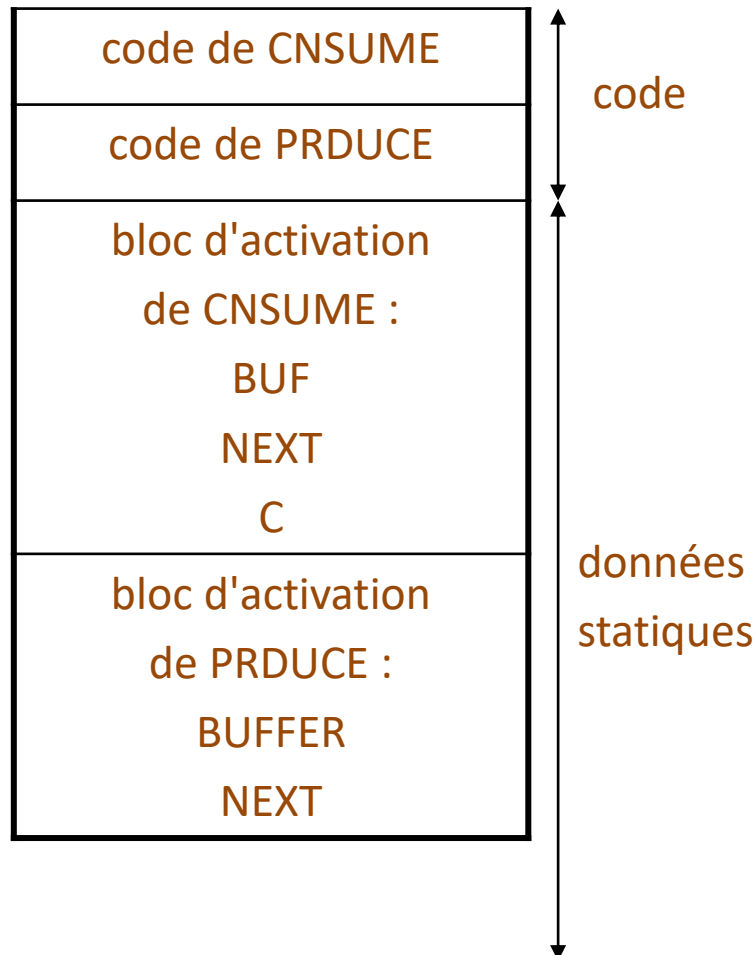
L'adresse est connue à la compilation

Si l'environnement associe une occurrence du nom *id* à l'adresse *a*, on dit que *id* est lié à *a*

La valeur stockée à une adresse dépend de chaque exécution

Une adresse peut contenir plusieurs valeurs successivement

Allocation exclusivement statique (Fortran 77)



La stratégie d'allocation mémoire la plus simple

Toutes les adresses sont statiques (constantes pendant toute l'exécution)

L'emplacement des blocs d'activation est statique

Les valeurs des noms locaux peuvent persister d'un appel au suivant (déclaration SAVE)

Limitations

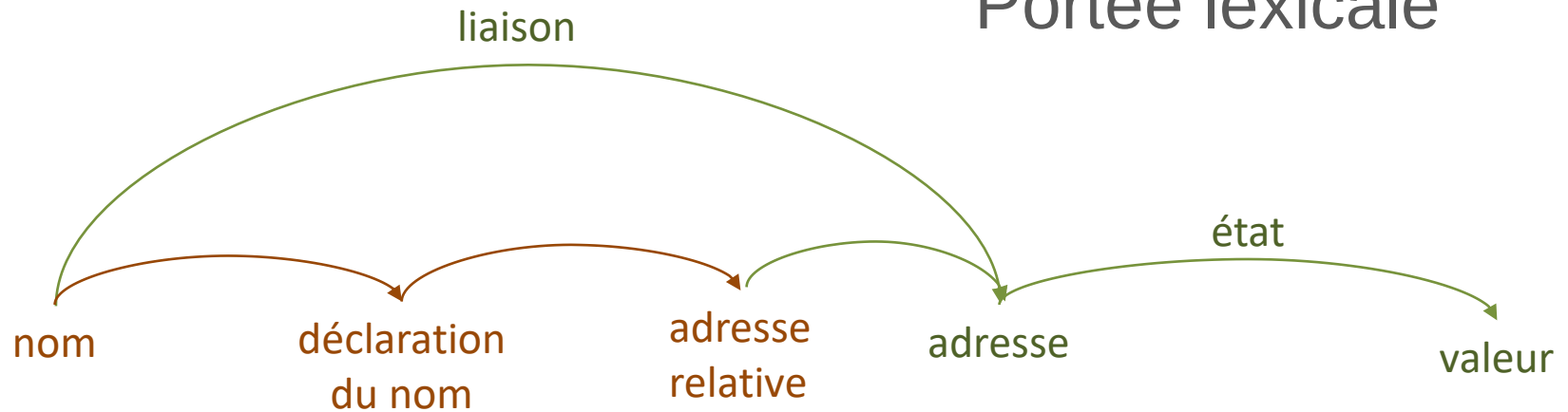
La taille de toutes les données est statique

La récursivité est impossible

L'allocation dynamique est impossible

Allocation automatique

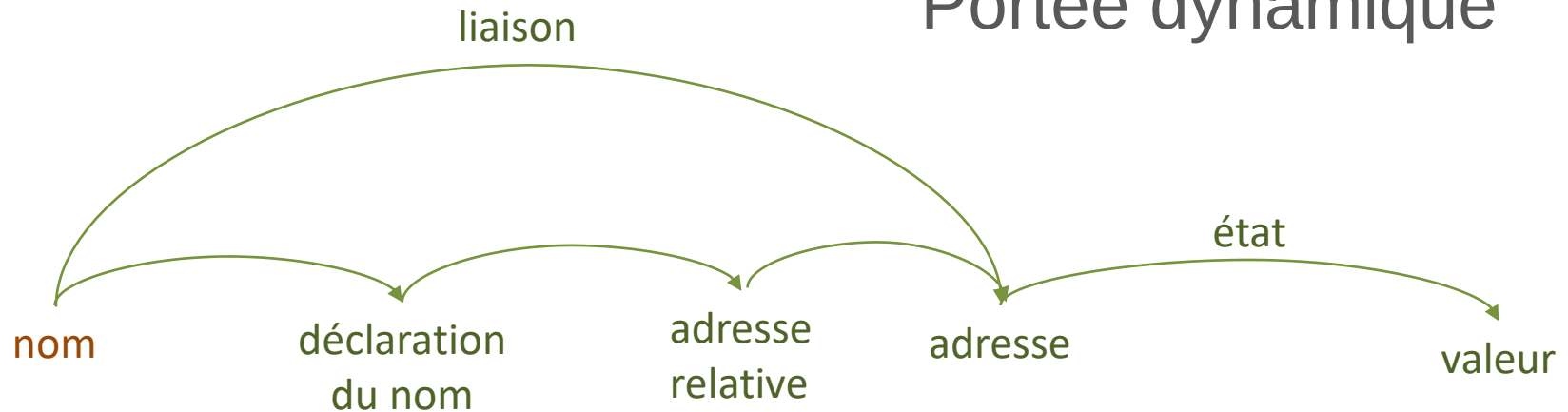
Portée lexicale



L'adresse relative est connue à la compilation

Allocation automatique

Portée dynamique



L'adresse relative est connue seulement à l'exécution

Une liaison a la même durée de vie que l'activation correspondante

Placement des variables automatiques

Nom	Type	Adresse relative
c1	char	-1
c2	char	-2
i	int	-6
d1	char	-7
d2	char	-8

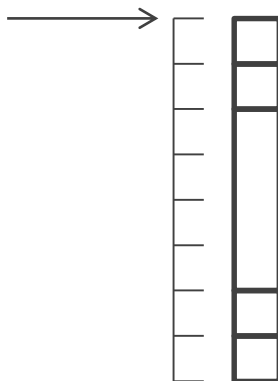
```
char c1,c2;
int i;
char d1,d2;
```

À la construction de la table des symboles
Attribuer les adresses relatives en tenant compte
des emplacements déjà occupés

Si les adresses relatives partent de 0 et sont
négatives, la dernière adresse relative
attribuée donne la taille occupée par les
variables allouées

On l'utilise pour allouer la mémoire

Base des
adresses
relatives



Calcul d'adresses en nasm

Nom	Type	Adresse relative
a	int	18

```
pop rax
mov dword [my_base+18], eax
```

Variables statiques

Base des adresses relatives : définie dans une section `.data` ou `.bss`

a:=tmp7

Variables automatiques

Base des adresses relatives : `rbp`

b:=tmp12

Nom	Type	Adresse relative
b	char	-5

```
pop rax
mov byte [rbp-5], al
```

Sommaire

Allocation des données

Portée des liaisons

Calcul des adresses

Amorçage

Faire un nouveau compilateur

$H \xrightarrow{L} B$

H : langage source, de haut niveau

B : langage cible, de bas niveau

Un compilateur est un programme écrit dans un langage

On l'écrit dans un langage (ici L)

Comment a-t-on fait le premier compilateur ?

Comment compiler le premier compilateur auto-hébergé d'un langage ?

Comment faire le premier compilateur natif qui écrit dans un langage donné ?

$H \xrightarrow{H} B$

Compilateur auto-hébergé :
écrit dans le langage qu'il compile

$H \xrightarrow{B} B$

Compilateur natif :
écrit dans son langage cible

Faire un nouveau compilateur

$H \xrightarrow{L} B$

H : langage source, de haut niveau

B : langage cible, de bas niveau

Un compilateur est un programme écrit dans un langage

On l'écrit dans un langage (ici L)

Comment écrire un compilateur en utilisant un compilateur existant ?

Le nouveau compilateur

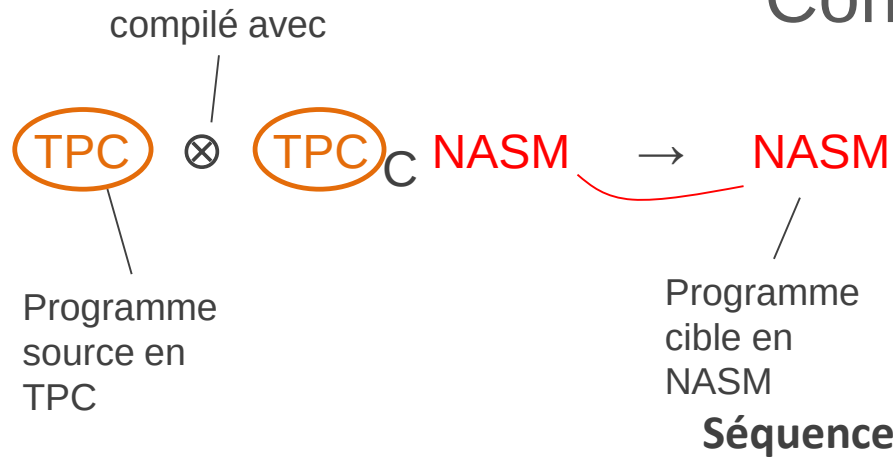
- soit sera écrit dans un autre langage (réécriture)
- soit compilera un autre langage de haut niveau (création d'un langage)
- soit compilera dans un autre langage de bas niveau (migration)

$H \xrightarrow{L'} B$

$H' \xrightarrow{L} B$

$H \xrightarrow{L} B'$

Combiner des compilateurs



$$\text{TPC}_C \text{ AA} \quad \text{AA}_C \text{ NASM} \rightarrow \text{TPC}_C \text{ NASM}$$

Compiler un compilateur

Bison (YACC)

$$\text{TPC} (\text{R}) \text{ AA} \quad \otimes \quad (\text{R})_C \text{ C} \rightarrow \text{TPC}_C \text{ AA}$$

$$\text{TPC}^{\text{gcc}} (\text{C}) \text{ NASM} \quad \otimes \quad (\text{C})_C \text{ X86} \rightarrow \text{TPC}_{\text{X86}} \text{ NASM}$$

Compilateur auto-hébergé (*self-hosting, self-compiling*)

C_C NASM

C_C X86

Compilateur d'un langage écrit dans ce même langage

Comment compiler un compilateur $H_H B$ (amorçage) ?

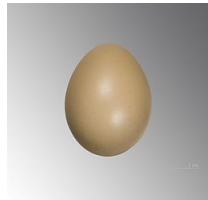
Quand on crée H , il faut un autre compilateur de H en B (l'amorce)

$H_H B \otimes H_L B \rightarrow H_B B$

Si L est un langage de haut niveau, il a fallu compiler $H_L B$ d'abord

$$H_H B \otimes H_L B \rightarrow H_B B$$

By 4028mdk09 (Own work) [CC BY-SA 3.0
(<http://creativecommons.org/licenses/by-sa/3.0/>)], via
Wikimedia Commons



By Didier Descouens (Own work) [CC BY-SA 3.0
(<http://creativecommons.org/licenses/by-sa/3.0/>)],
via Wikimedia Commons

$$H_B B$$


By Unknown (Smithsonian Institution) (Flickr: Grace Hopper and UNIVAC) [CC BY 2.0
(<http://creativecommons.org/licenses/by/2.0/>)], via Wikimedia Commons

Compiler un compilateur

Comment a-t-on créé le premier compilateur ?
En langage machine, à la main, 1952
Alick Glennie, à l'Université de Manchester
Grace Hopper, chez un fabricant de machines à
écrire américain

Créer un langage

Écrire le premier compilateur d'un nouveau langage K

On ne peut pas l'écrire en K dès le départ

$K_H B$ puis $K_K B$: seule la partie avant change

Comment éviter d'écrire deux compilateurs complets ?

- version minimale K_0 écrite en H
- version K_0 réécrite en K_0
- version plus étendue K_1 écrite en K_0
- ...
- version finale K écrite en K_{n-1}

Plusieurs versions du compilateur (auto-amorçage)

Versions du langage

Le premier compilateur complet auto-hébergé

de K est $K_{K_{n-1}} B$

$$K_H B \otimes H_B B \rightarrow K_B B$$

$$K_0 H B \otimes H_B B \rightarrow K_0 B B$$

$$K_0 K_0 B \otimes K_0 B B \rightarrow K_0 B B$$

$$K_1 K_0 B \otimes K_0 B B \rightarrow K_1 B B$$

...

$$K_{K_{n-1}} B \otimes K_{n-1} B B \rightarrow K_B B$$

Compilateur natif

C X_{86} X_{86}

Compilateur écrit dans le même langage que son langage cible

Il s'exécute sur l'architecture pour laquelle il produit du code

Migration

Écrire un compilateur $H_H B'$ pour un nouveau langage cible B'

Partir d'un compilateur existant $H_H B$

Seule la partie arrière change

Comment compiler $H_H B'$ en un compilateur natif ?

$H_H B$

$H_H B'$

Faire migrer un compilateur d'une architecture vers une autre

$H_H B'$ $H_{B'} B'$

Compiler $H_H B'$ en un compilateur natif
Il faut le compiler deux fois

Première compilation

On obtient un **compilateur croisé** (*cross compiler*) :

$H_H B' \otimes H_B B \rightarrow H_B B'$

- exécutable sur une architecture,
- produit du code exécutable sur une autre

$H_H B' \otimes H_B B' \rightarrow H_{B'} B'$

Deuxième compilation

Toujours sur l'architecture B mais avec le compilateur croisé

Copier le compilateur obtenu sur la machine cible

Merci

CONTACT

ERIC LAPORTE

00 +33 (0)1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR